ASSIGNMENT – 3

Software Testing and Quality Assurance

MCA – 2nd Year

Prepared Answers

Name: _____

Roll No.: _____

Department: _____

1. What is Test Planning? Explain the main steps involved in the test planning process.

Test planning is the process of deciding what will be tested, how testing will be performed, who will perform it, and when it will be completed. It helps testing activities remain organized and ensures that the required time and resources are available.

Main steps:

1. Understand requirements: Study the software requirements and identify what needs to be tested.
2. Define the scope: Decide which features are included in testing and which are excluded.
3. Identify the test objectives: Set clear goals, such as checking functionality, performance, and reliability.
4. Select the test approach: Decide the testing levels, types, tools, and techniques to be used.
5. Estimate resources: Identify the people, hardware, software, and tools required.
6. Prepare the schedule: Set timelines, milestones, and responsibilities.
7. Identify risks: List possible problems, such as delays or unavailable test environments, and plan responses.
8. Define entry and exit criteria: Specify when testing can start and when it can be considered complete.
9. Prepare and review the test plan: Document the decisions, obtain approval, and update the plan when necessary.

2. What is a Test Strategy? Explain its importance in software testing.

A test strategy is a high-level document that describes the general approach an organization or project will follow to test software. It defines the testing principles, levels, types, methods, tools, and quality goals.

Importance:

- It provides a common direction for the testing team.
- It helps select suitable testing methods and tools.
- It supports consistent testing across the project.
- It helps identify major risks and focus on critical areas.
- It improves the use of time, budget, and people.
- It helps stakeholders understand how software quality will be evaluated.

3. What is a Test Plan? Explain the major contents of a Test Plan.

A test plan is a document that describes the scope, objectives, resources, schedule, and activities for testing a particular software project.

Major contents:

1. Test plan identifier and introduction.
2. Objectives and scope of testing.
3. Features to be tested and features not to be tested.
4. Test approach, levels, and types of testing.
5. Test environment, including hardware, software, and test data.

6. Roles and responsibilities of team members.
7. Test deliverables, such as test cases, scripts, and reports.
8. Schedule, milestones, and estimated effort.
9. Entry and exit criteria.
10. Risks, assumptions, dependencies, and contingency plans.
11. Defect reporting and tracking process.
12. Approval and review details.

4. Differentiate between a Test Strategy and a Test Plan.

A test strategy gives the overall approach to testing, while a test plan explains how testing will be carried out for a particular project or testing activity.

Test Strategy:

- High-level and broad in scope.
- Focuses on the overall testing approach and principles.
- Often applies across projects or to a whole program.
- Usually changes less frequently.
- Defines general methods, standards, and tools.

Test Plan:

- More detailed and project-specific.
- Focuses on tasks, people, timelines, and deliverables.
- Applies to a specific project, release, or test level.
- Updated as project requirements or schedules change.
- Defines scope, responsibilities, schedule, resources, and criteria.

In short, the strategy explains the general approach; the plan explains the specific execution of testing.

5. What are the main goals and objectives of software testing? Explain.

The main goal of software testing is to evaluate software and find defects before the software is released. Testing also provides information about quality and reduces the risk of failure.

Objectives:

1. Detect defects: Find errors, faults, and unexpected behaviour.
2. Verify requirements: Check whether the software meets specified requirements.
3. Validate user needs: Check whether the software is suitable for its intended use.
4. Improve quality: Evaluate reliability, usability, security, performance, and compatibility.
5. Prevent failures: Reduce the chance of problems after release.
6. Reduce risk: Identify critical problems that could affect users or business operations.
7. Build confidence: Provide useful evidence about the quality of the product.
8. Support decisions: Help stakeholders decide whether the software is ready for release.

Testing cannot prove that software has no defects; it helps discover defects and assess risk.

6. What are Test Phases? Explain the different phases of software testing.

Test phases are stages through which testing activities are planned, prepared, executed, and completed. The common testing levels are:

- Unit Testing: Individual functions, methods, or components are tested, usually by developers.
- Integration Testing: Combined modules are tested to check whether they communicate and exchange data correctly.
- System Testing: The complete integrated application is tested against its requirements.
- Acceptance Testing: Users, customers, or business representatives check whether the software meets their needs and is ready for use.

Supporting activities usually occur across these phases: test planning, test analysis and design, test implementation, test execution, defect reporting, retesting, regression testing, and test closure. These activities may overlap rather than follow a strict sequence.

7. What factors should be considered while developing a Test Strategy?

The following factors should be considered:

- Project scope, objectives, and requirements.
- Type, size, and complexity of the software.
- Business importance and risks of failure.
- Applicable laws, standards, and security requirements.
- Testing levels and types required.
- Available budget, time, skills, and team size.
- Tools, automation opportunities, and existing frameworks.
- Availability of test environments and test data.
- Interfaces with other systems and external dependencies.
- Expected performance, usability, reliability, and compatibility.
- Defect management and reporting procedures.
- Entry and exit criteria and the method for measuring quality.

8. Explain the resource requirements needed for software testing.

Resource requirements are the people, tools, equipment, and other facilities needed to perform testing effectively.

- Human resources: Test manager, test lead, testers, automation engineers, developers, business analysts, and subject experts.
- Hardware: Computers, servers, mobile devices, network equipment, and storage.
- Software and tools: Test management tools, defect tracking tools, automation frameworks, browsers, and monitoring tools.
- Test environment: Separate development, testing, staging, or other required environments with suitable configurations.
- Test data: Valid, invalid, boundary, and realistic sample data. Sensitive data should be protected or masked.
- Time and budget: Enough time and funding for planning, test design, execution, defect fixing, retesting, and reporting.

- Documentation: Requirements, specifications, test plans, test cases, and user guides.
- Training and support: Training on the product, tools, domain, and testing methods.

Correct resource planning prevents delays and helps achieve the required level of quality.

9. What is a Test Schedule? Explain the importance of scheduling in software testing.

A test schedule is a timetable showing testing activities, their start and finish dates, dependencies, milestones, and responsible team members.

Importance:

- It helps complete testing within the project deadline.
- It assigns work and responsibilities clearly.
- It identifies dependencies, such as waiting for a stable build.
- It helps allocate resources efficiently.
- It allows progress to be monitored against planned dates.
- It highlights delays early so corrective action can be taken.
- It provides stakeholders with a clear view of testing progress.

A good schedule should include time for test design, environment setup, execution, defect fixing, retesting, regression testing, and final reporting.

10. What is a Test Case? Explain the main components of a test case.

A test case is a documented set of conditions, inputs, steps, and expected results used to check whether a particular feature works correctly.

Main components:

1. Test Case ID: A unique identifier.
2. Test Case Title: A short description of what is being checked.
3. Objective: The purpose of the test.
4. Preconditions: Conditions that must be met before execution.
5. Test Data: Inputs needed for the test.
6. Test Steps: Instructions to perform the test.
7. Expected Result: The result that should occur.
8. Actual Result: The result observed during execution.
9. Status: Pass, fail, blocked, or not run.
10. Priority: The importance or urgency of the test.
11. Environment: The device, operating system, browser, or build used.
12. Remarks/Defect ID: Notes or a link to a reported defect.

11. Explain the process of designing and writing effective test cases.

Effective test cases should be clear, complete, repeatable, and linked to requirements.

Steps:

1. Study the requirements and understand the expected behaviour.

2. Identify test conditions, features, and possible risks.
3. Choose suitable design techniques, such as equivalence partitioning and boundary value analysis.
4. Define preconditions and prepare test data.
5. Write simple, numbered steps that another tester can follow.
6. Specify measurable expected results for each case.
7. Include positive cases, negative cases, and boundary conditions.
8. Give each test case a unique ID and link it to the relevant requirement.
9. Review the cases for clarity, coverage, duplication, and missing scenarios.
10. Execute the test cases, record results, and update them when requirements change.

Example: For a login form, test valid credentials, an incorrect password, empty fields, and locked or inactive accounts if those behaviours are specified.

12. What is Bug Reporting? Explain the important information included in a bug report.

Bug reporting is the process of recording a software defect in a tracking system so that the team can understand, reproduce, prioritize, and fix it.

Important information:

- Bug ID: A unique reference number.
- Title/Summary: A short description of the problem.
- Description: What went wrong.
- Steps to reproduce: Exact actions that cause the problem.
- Expected result: What should have happened.
- Actual result: What happened instead.
- Environment: Build, operating system, browser, device, and other relevant details.
- Severity: The impact of the defect on the system.
- Priority: How urgently it should be fixed.
- Attachments: Screenshots, logs, videos, or error messages.
- Reporter and date: Who reported it and when.
- Status and assignee: Current state and person responsible.

A good report is factual, clear, concise, and detailed enough for another person to reproduce the defect.

13. Explain the Bug Life Cycle in software testing.

The bug life cycle is the sequence of states a defect passes through from reporting to closure. The exact states vary between teams.

Common stages:

1. New: A tester reports a defect.
2. Assigned: The defect is assigned to a developer or responsible team.
3. Open: The team reviews and investigates it.

4. Fixed/Resolved: The developer makes a change and marks it as fixed.
5. Retesting: The tester checks whether the fix works.
6. Verified: The tester confirms that the defect has been corrected.
7. Closed: The defect is considered complete.

Other possible states:

- Reopened: The problem still occurs after the fix.
- Rejected/Invalid: The report is not considered a valid defect.
- Duplicate: The same defect has already been reported.
- Deferred: The fix is postponed to a later release.
- Cannot Reproduce: The team cannot reproduce the issue with the provided information.

A defect may move backward or take a different path depending on the investigation and retest results.

14. What are Test Metrics? Explain their importance in measuring testing activities.

Test metrics are measurable values used to evaluate the progress, coverage, effectiveness, and quality of testing.

Importance:

- They show how much testing has been completed.
- They help identify areas with insufficient coverage.
- They support planning and estimation.
- They help track defects and recurring problems.
- They allow teams to compare actual progress with targets.
- They provide information for release decisions and process improvement.

Metrics should be interpreted in context. A high number of test cases executed does not automatically mean that the product is high quality.

15. Explain different types of testing metrics used in software testing.

Common testing metrics include:

1. Test Case Execution: Number of test cases executed compared with the total planned.
2. Test Pass Rate: Percentage of executed tests that passed.

Formula: Pass Rate = (Passed Test Cases ÷ Executed Test Cases) × 100.

3. Test Coverage: The extent to which requirements, features, code, or risks have been tested.
4. Defect Density: Number of defects relative to the size of a software component, such as defects per thousand lines of code.
5. Defect Detection Rate: Number of defects found during a particular period or testing phase.
6. Defect Leakage: Defects discovered after release that were not found during earlier testing.
7. Defect Removal Efficiency: Percentage of defects removed before release out of all defects found before and after release.
8. Test Case Productivity: Number of test cases designed or executed per person or time period.

9. Mean Time to Repair: Average time required to fix a defect or restore service, depending on the definition used.
10. Schedule Variance: Difference between planned and actual testing progress or completion dates.

Metrics are most useful when definitions are consistent and results are reviewed alongside risk, severity, and project context.

16. What are Test Case Design Techniques? Explain their importance in software testing.

Test case design techniques are systematic methods used to select test inputs and conditions. They help testers create effective tests without checking every possible input.

Common techniques:

- Equivalence Partitioning: Divides input data into groups expected to behave similarly.
- Boundary Value Analysis: Tests values at and around the edges of allowed ranges.
- Decision Table Testing: Checks combinations of conditions and their resulting actions.
- State Transition Testing: Tests how a system behaves when it moves between states.
- Use Case Testing: Creates tests based on user workflows.
- Error Guessing: Uses experience to predict likely mistakes.

Importance:

These techniques improve coverage, reduce unnecessary test cases, help find defects efficiently, and make test design more systematic and repeatable.

17. What is Equivalence Partitioning? Explain with a suitable example.

Equivalence Partitioning is a black-box test design technique that divides input values into groups, or partitions, expected to be processed in the same way. A representative value from each partition is selected for testing.

Example: Suppose an application accepts an age from 18 to 60, inclusive.

Partitions:

- Invalid: age below 18.
- Valid: age from 18 to 60.
- Invalid: age above 60.

Representative test values:

- 15 — expected to be rejected.
- 35 — expected to be accepted.
- 65 — expected to be rejected.

This technique reduces the number of tests because every value in a partition is assumed to behave similarly. That assumption should be checked against the requirements and risks.

18. What is Boundary Value Analysis? Explain with a suitable example.

Boundary Value Analysis (BVA) is a test design technique that focuses on values at the edges of an allowed range, because errors often occur near boundaries.

Example: Suppose an application accepts an age from 18 to 60, inclusive.

Useful test values:

- 17 — just below the minimum; expected to be rejected.
- 18 — minimum allowed value; expected to be accepted.
- 19 — just above the minimum; expected to be accepted.
- 59 — just below the maximum; expected to be accepted.
- 60 — maximum allowed value; expected to be accepted.
- 61 — just above the maximum; expected to be rejected.

BVA is useful for fields such as age, marks, quantity, date ranges, and password length. The expected result must follow the stated requirements.

19. Differentiate between Equivalence Partitioning and Boundary Value Analysis.

Equivalence Partitioning (EP):

- Divides inputs into groups expected to behave similarly.
- Selects representative values from each group.
- Focuses on categories of input.
- Example for age 18–60: test 15, 35, and 65.

Boundary Value Analysis (BVA):

- Focuses on the edges of allowed and disallowed ranges.
- Selects values at, just below, and just above boundaries.
- Focuses on possible errors near limits.
- Example for age 18–60: test 17, 18, 19, 59, 60, and 61.

Both are black-box test design techniques. EP gives broad coverage of input groups, while BVA provides detailed checking around range limits. They are often used together.

20. What is Test Case Planning? Explain the steps involved in test case planning.

Test case planning is the process of deciding which test cases are needed, how they will cover the requirements, what data and environment they need, and how they will be prepared and executed.

Steps:

1. Review requirements: Understand the features and expected behaviour.
2. Identify test conditions: List the functions, scenarios, and risks that need testing.
3. Define coverage: Map test conditions to requirements to avoid gaps.
4. Select design techniques: Use suitable methods such as equivalence partitioning, boundary value analysis, and decision tables.
5. Prioritize test cases: Give higher priority to critical features and high-risk areas.
6. Identify test data: Prepare valid, invalid, and boundary inputs.
7. Confirm preconditions and environment: Specify accounts, configuration, devices, and software needed.
8. Estimate effort and assign responsibility: Decide who will design, review, and execute the tests.

9. Write and review test cases: Include steps and clear expected results; check for clarity and completeness.
10. Organize and maintain: Store cases with IDs, link them to requirements, and update them when the software changes.

Good test case planning improves coverage, saves time, and makes testing easier to track and repeat.